

Activity 12 - Integrate components into a tested command-line tool

AI Vibe Coding with Rust / TGS-2023039924 / version 3.0 / K4/A4

Specification

`run(csv, threshold) -> Result<String, String>` returns "ITEMS n\nTOTAL \$D.CC\nLOW STOCK a, b | none". The binary takes `<file> [threshold=5]`; exit 0 on success, 1 for unreadable or invalid data, 2 for usage errors.

Plan before code

Act as a senior Rust reviewer. Specification: `run(csv, threshold) -> Result<String, String>` returns "ITEMS n\nTOTAL \$D.CC\nLOW STOCK a, b | none". The binary takes `<file> [threshold=5]`; exit 0 on success, 1 for unreadable or invalid data, 2 for usage errors. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

Implement "Integrate components into a tested command-line tool" in `starter/src` so that the supplied tests pass. Keep `main` thin; put every rule in `lib.rs` where the library tests can reach it. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

Here is my exact cargo output: `<paste the first error or failing test>`. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

Review checklist

Signatures unchanged; no unsafe; no `unwrap()` on input; no new crates; every boundary tested; cargo test, `clippy -D warnings` and `fmt --check` pass; you can explain every line.